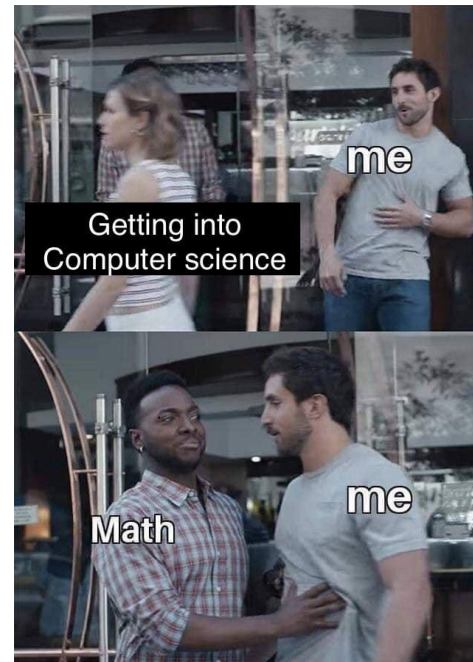


D1: Asymptotic Notation and Proof Methods Review



Sec 17: Fr 11:30am-12:30pm FXB 1024
IA: Pearl Lin

Agenda

- ▶ Introduction
- ▶ Asymptotic notation
- ▶ Proof methods review
- ▶ Potential method
- ▶ Some worksheet problems

Introduction

About Me

- ▶ Pearl Lin
- ▶ Senior in computer science
- ▶ 3rd semester w/ 376
- ▶ Hobbies
 - ▶ Knitting, piano, clam digging, crochet, Stardew, watching dramas
- ▶ Office hours (starting next week)
 - ▶ Mon 4:30-6:30PM, Leinweber 2421
 - ▶ Wed 1-4PM, Leinweber 2421



About Discussion Attendance

- ▶ You may attend any section, but **please try to be consistent**
- ▶ If you attend at least 10 of 14 discussions throughout the semester, then you get 3% for free for your overall grade and your exam scores weigh 3% less
 - ▶ **FAQ:** What if I attend 9/10 discussions/workshops? 0-3% for discussion attendance is **all or nothing**, so you would receive 0%

Assignment	Weight
Homeworks	40%
Exams (midterm and final)	56-60%
Discussion attendance	0-3%
Course evaluations	0-1%
Total	100%

Asymptotic Notation

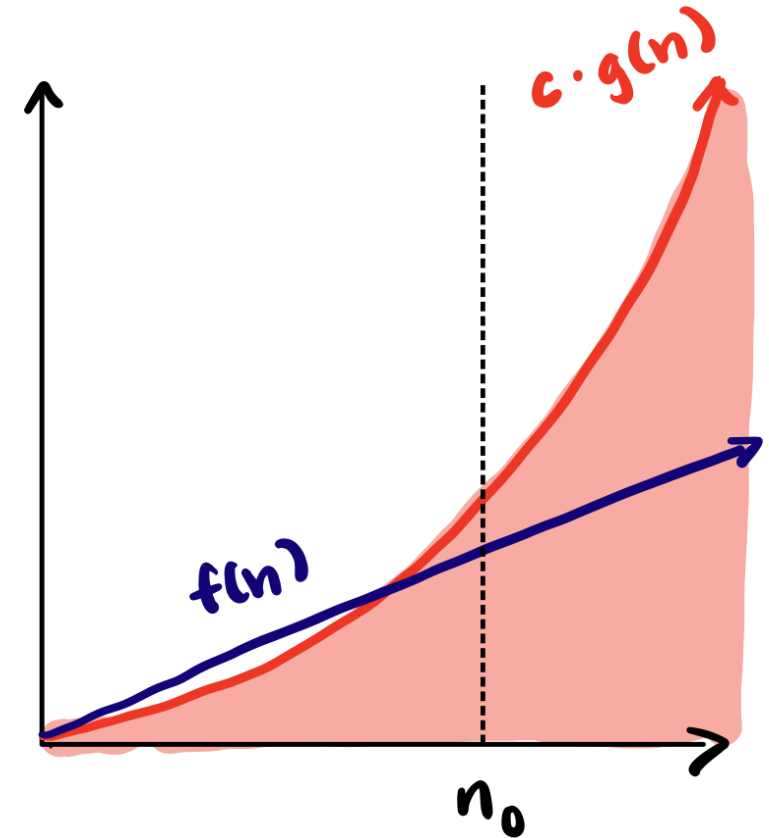
Big-O: Upper Bound

Useful: L'Hôpital's Rule

If $f(n), g(n)$ are differentiable, $g'(n) \neq 0$ for all large enough n , and $\lim_{n \rightarrow \infty} g(n) = \infty$:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)} \quad (\text{if } \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)} \text{ exists})$$

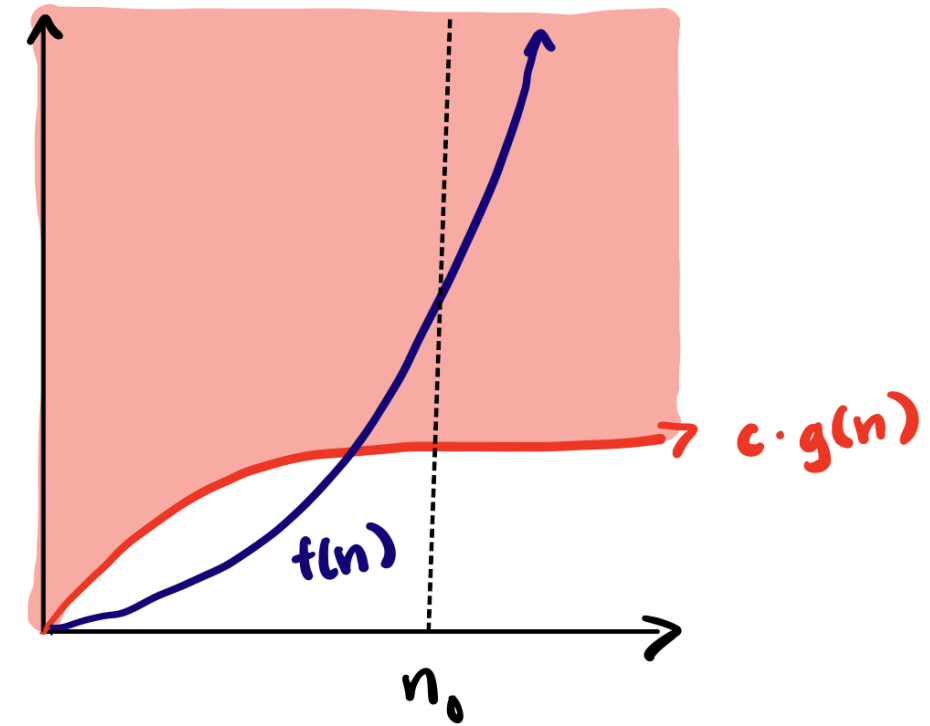
- ▶ Let $f(n)$ and $g(n)$ be positive functions
- ▶ **Definition:** $f(n) = O(g(n))$ if there exists positive constants $c, n_0 > 0$ such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$
- ▶ **Proof methods:**
 - ▶ When $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ exists or is infinite, show that $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ is finite (Lemma 4)
 - ▶ Find constants c, n_0 from definition



Big-Ω: Lower Bound

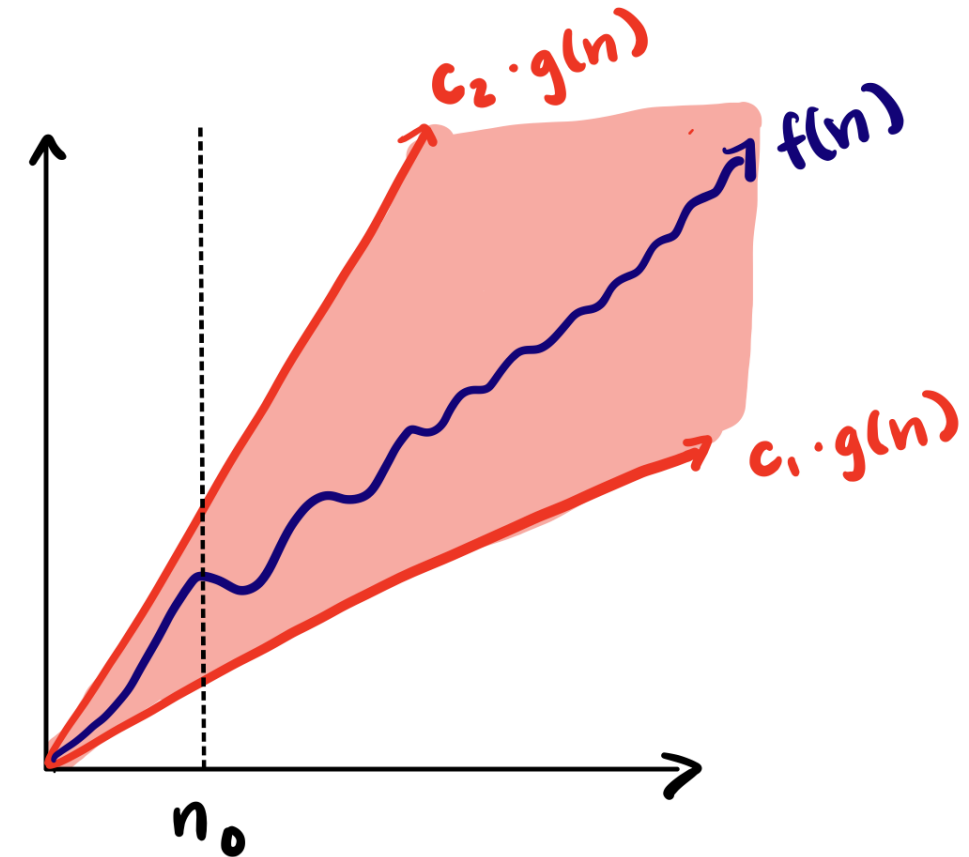
Note: Lemmas 4 and 5 can't be applied when the limit does not exist (not infinite). In these cases, prove using the definition.

- ▶ Let $f(n)$ and $g(n)$ be positive functions
- ▶ **Definition:** $f(n) = \Omega(g(n))$ if there exists positive constants $c, n_0 > 0$ such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$
- ▶ **Proof methods:**
 - ▶ When $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ exists or is infinite, show that $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} > 0$ (Lemma 5)
 - ▶ Find constants c, n_0 from definition



Big- Θ : “Exact” Bound

- ▶ Let $f(n)$ and $g(n)$ be positive functions
- ▶ **Definition:** $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$
 - ▶ Equivalently: $f(n) = \Theta(g(n))$ if there exists positive constants $c_1, c_2, n_0 > 0$ such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$
- ▶ **Proof methods:**
 - ▶ When $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ exists or is infinite, show that $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ is a **finite number** > 0 (Lemmas 4 + 5)
 - ▶ Find constants c_1, c_2, n_0 from definition



Proof Methods Review

Direct Proof ($P \Rightarrow Q$)

- ▶ Strategy: Assume P is true, then use a series of logical statements to show Q is true
- ▶ **Scaffolding:**
 - ▶ Assume P
 - ▶ ...
 - ▶ WTS (Want To Show): Q

Proof by Contrapositive ($\neg Q \Rightarrow \neg P$)

- ▶ Strategy: Directly prove the contrapositive $\neg Q \Rightarrow \neg P \equiv P \Rightarrow Q$
- ▶ **Scaffolding:**
 - ▶ We will prove the contrapositive $\neg Q \Rightarrow \neg P$
 - ▶ Assume $\neg Q$
 - ▶ ...
 - ▶ WTS: $\neg P$

Proof by Contradiction ($\neg P \equiv F$)

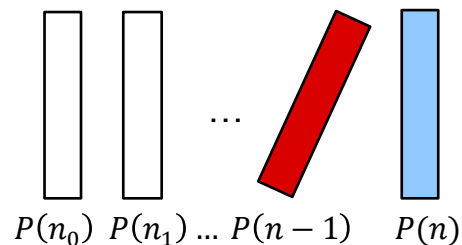
- ▶ Strategy: Assume the negation, then arrive at a logical contradiction
- ▶ **Scaffolding:**
 - ▶ Seeking a contradiction, assume the negation $\neg P$
 - ▶ ...
 - ▶ WTS: We have found the contradiction _____. So indeed $P \equiv T$.

Proof by Induction

- ▶ Strategy: Show that $P(n)$ is true for some starting number(s), and that previous $P(n)$ being true implies that the current $P(n)$ is true. Then the “truthiness” of $P(n)$ will eventually propagate to all n

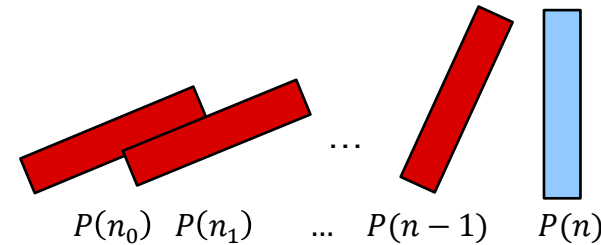
- ▶ **Scaffolding** (weak induction):

- ▶ Base case(s):
 - ▶ Prove $P(n_0)$
- ▶ Inductive step:
 - ▶ Assume $P(n - 1)$
 - ▶ ...
 - ▶ WTS: $P(n)$



- ▶ **Scaffolding** (strong induction):

- ▶ Base case(s):
 - ▶ Prove $P(n_0)$
- ▶ Inductive step:
 - ▶ Assume $P(n_0) \wedge P(n_1) \dots \wedge P(n - 1)$
 - ▶ ...
 - ▶ WTS: $P(n)$



Potential Method

Potential Method



- Strategy: Show that an algorithm is always making progress towards finishing. Then upper bound the # of steps before which it must halt

- Potential function:

- $s: A \rightarrow \mathbb{R}$
 - maps a function's state to a finite, real number (so we can keep track of how far it has progressed)
- Requirements:
 - 1. $s_i \geq l$ for some $l \in \mathbb{R}$ (lower bound)
 - 2. $s_{i+1} - s_i \leq c < 0$ for some $c \in \mathbb{R}$ (decreases by a fixed amount each iteration)

- Scaffolding:

- Define a potential function s
- Show that s satisfies both requirements
- max # of iterations is finite:

- $$\frac{\text{amount of work}}{\text{min work done each iteration}} = \frac{s_0 - l}{c}$$

Note: This is not always the tightest bound! See Euclid example from class.

Worksheet

Q1

1. Comparing asymptotic running times.

On the same computer with the same input, an algorithm having $O(n \log n)$ worst-case running time runs in less time than **all** **some** **no** algorithms having $O(n^2)$ worst-case running time.

$$T_A(n) = n$$

$$T_B(n) = n \log n$$

$$n_0 = 5$$

$$T_A(n) = n \log n \quad \checkmark$$

$$T_B(n) = n \quad \checkmark$$

Q1 (annotations)

1. Comparing asymptotic running times.

On the same computer with the same input, an algorithm having $O(n \log n)$ worst-case running time runs in less time than **all/some/no** algorithms having $O(n^2)$ worst-case running time.

Q1 (solution)

1. Comparing asymptotic running times.

On the same computer with the same input, an algorithm having $O(n \log n)$ worst-case running time runs in less time than **all/some/no** algorithms having $O(n^2)$ worst-case running time.

- ▶ **Some.** Consider the following candidates for Algorithm A = $O(n \log n)$ and Algorithm B = $O(n^2)$
- ▶ Algorithm A is worse:
 - ▶ $T_A(n) = 500n \log n$
 - ▶ $T_B(n) = n$
 - ▶ Take $n > 50$, then $T_B(n) \leq T_A(n)$
- ▶ Algorithm A is better:
 - ▶ $T_A(n) = n \log n$
 - ▶ $T_B(n) = 600n^2$
 - ▶ Take $n > 50$, then $T_A(n) \leq T_B(n)$

Q2A

2. Asymptotics.

For the following pairs of functions $f(n)$ and $g(n)$, is $f(n) = O(g(n))$? Justify your answer by applying the definition of big-O or by applying a limit argument.

- $f(n) = \log_a n$, $g(n) = \log_b n$ for constants $a, b > 1$.

Hint: rewrite f and g using change of base rule, then use the limit argument

$$\left. \begin{aligned} f(n) &= \log_a(n) = \frac{\log_{10}(n)}{\log_{10}(a)} \\ g(n) &= \log_b(n) = \frac{\log_{10}(n)}{\log_{10}(b)} \end{aligned} \right\}$$

finite

$$\begin{aligned} f(n) &= O(g(n)) \\ \lim_{n \rightarrow \infty} \left[\frac{f(n)}{g(n)} \right] &= \lim_{n \rightarrow \infty} \frac{\log_{10}(n)}{\log_{10}(n)} \cdot \frac{\log_{10}(b)}{\log_{10}(a)} \\ &= \lim_{n \rightarrow \infty} \left[\frac{\log_{10}(b)}{\log_{10}(a)} \right] = \frac{\log_{10}(b)}{\log_{10}(a)} \end{aligned}$$

lim
4.
e.

Q2A (annotations)

2. Asymptotics.

For the following pairs of functions $f(n)$ and $g(n)$, is $f(n) = O(g(n))$? Justify your answer by applying the definition of big-O or by applying a limit argument.

- $f(n) = \log_a n$, $g(n) = \log_b n$ for constants $a, b > 1$.

Q2A (solution)

2. Asymptotics.

For the following pairs of functions $f(n)$ and $g(n)$, is $f(n) = O(g(n))$? Justify your answer by applying the definition of big-O or by applying a limit argument.

- $f(n) = \log_a n$, $g(n) = \log_b n$ for constants $a, b > 1$.

► **Yes.**

► Using limit argument, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{\log_a n}{\log_b n} = \lim_{n \rightarrow \infty} \frac{\ln(n)\ln(b)}{\ln(n)\ln(a)} = \frac{\ln(b)}{\ln(a)}$ (finite)

► By Lemma 4, $f(n) = O(g(n))$

Q3B

$$Q \rightarrow P$$

3. Faulty proofs.

For each of the proposed “proofs” of the following (true!) statements, determine what is incorrect about the proof.

(b) Claim: $\overbrace{\text{If } n^2 \text{ is even, then } n \text{ is even.}}^P$ Proof: $\overbrace{\text{If } n \text{ is even, then } n = 2k \text{ for some integer } k.}^Q$
Therefore, $\underbrace{n^2 = 4k^2 = 2 \cdot 2k^2}_{P} \text{ is even.}$

proved converse,
not original statement.

Q3B (annotations)

3. Faulty proofs.

For each of the proposed “proofs” of the following (true!) statements, determine what is incorrect about the proof.

- (b) Claim: If n^2 is even, then n is even. Proof: If n is even, then $n = 2k$ for some integer k . Therefore, $n^2 = 4k^2 = 2 \cdot 2k^2$ is even.

Q3B (solution)

3. Faulty proofs.

For each of the proposed “proofs” of the following (true!) statements, determine what is incorrect about the proof.

(b) Claim: If n^2 is even, then n is even. Proof: If n is even, then $n = 2k$ for some integer k . Therefore, $n^2 = 4k^2 = 2 \cdot 2k^2$ is even.

► Proves the converse instead of the contrapositive (not an equivalent statement)

Q8A

Hint: start w/ an initial guess that is a simple combination of x and y , then modify it

8. Potential functions.

Suppose *input* is a function that returns a user-specified positive integer. For each of the following problems, do the following tasks:

- Determine whether the following program halts for any sequence of (valid) inputs.
- If so, provide a proof using the potential method; if not, describe a sequence of inputs that causes the program to never halt.

(a)

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
```

Q8A (annotations)

(a)

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
```

Q8A (solution)

$x \leftarrow y$

(a)

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
```

This program halts. A valid potential function is $s = 2x + y$.

We will show that s satisfies Requirement #1 (lower bound). x and y will never be negative in any state of the algorithm since the algorithm halts immediately after either x or y attains a value of 0. Thus, $2x + y \geq 0$ and we have $l = 0$ as the lower bound.

We will show that s satisfies Requirement #2 (decreases by at least a fixed amount each iteration). There are 2 possible cases per iteration:

Case 1: z is even. Then $x_{i+1} = x_i - 1$ and $y_{i+1} = y_i + 1$. Given that $s_{i+1} = 2x_{i+1} + y_{i+1}$ and $s_i = 2x_i + y_i$, we have that $s_{i+1} = 2(x_i - 1) + y_i + 1 = 2x_i - 2 + y_i + 1 = 2x_i + y_i - 1 = s_i - 1$. So $s_{i+1} - s_i \leq -1$.

Case 2: z is odd. Then $x_{i+1} = x_i$ and $y_{i+1} = y_i - 1$. $s_{i+1} = 2(x_i) + y_i - 1 = 2x_i + y_i - 1 = s_i - 1$. Thus, $s_{i+1} - s_i \leq -1$.

Choose $c = -1$. $s_{i+1} - s_i \leq -1$ is satisfied for all cases.

Q8B

8. Potential functions.

Suppose *input* is a function that returns a user-specified positive integer. For each of the following problems, do the following tasks:

- Determine whether the following program halts for any sequence of (valid) inputs.
- If so, provide a proof using the potential method; if not, describe a sequence of inputs that causes the program to never halt.

(b)

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
10:     $x \leftarrow x + 1$  // This line differs from the program in (a)
```

Q8B (annotations)

(b)

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
10:     $x \leftarrow x + 1$  // This line differs from the program in (a)
```

Q8B (solution)

This program **doesn't halt** for all inputs. A sequence of inputs that causes a loop could be any alternating sequence of even z and odd z .

(b)

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
10:     $x \leftarrow x + 1$  // This line differs from the program in (a)
```

Additional Slides

Properties of Asymptotic Notations

Consider the positive-bounded functions $f_1(n) = \Theta(g_1(n))$ and $f_2(n) = \Theta(g_2(n))$

Addition rule: take the dominant function

$$(f_1 + f_2)(n) = \Theta(\max(g_1(n), g_2(n)))$$

Scalar multiplication: doesn't change

$$(c \cdot f_1)(n) = \Theta(g_1(n))$$

Product: multiply the runtimes

$$(f_1 \cdot f_2)(n) = \Theta(g_1(n) \cdot g_2(n))$$

Example of Big-O

$$f(n) = n, g(n) = n \log n$$

► Limit argument:

► $\lim_{n \rightarrow \infty} \frac{n}{n \log n} = \frac{\infty}{\infty}$

► Since $g(n) = n \log n$ is a smooth function (and thus differentiable), $\lim_{n \rightarrow \infty} n \log n = \infty$, and $g'(n) = \log n + \frac{1}{\ln 10} > 0$ for $n > 1$, we can use L'Hôpital's Rule.

► $\lim_{n \rightarrow \infty} \frac{n}{n \log n} = \lim_{n \rightarrow \infty} \frac{1}{\log n + \frac{1}{\ln 10}}$ by L'Hôpital's Rule.

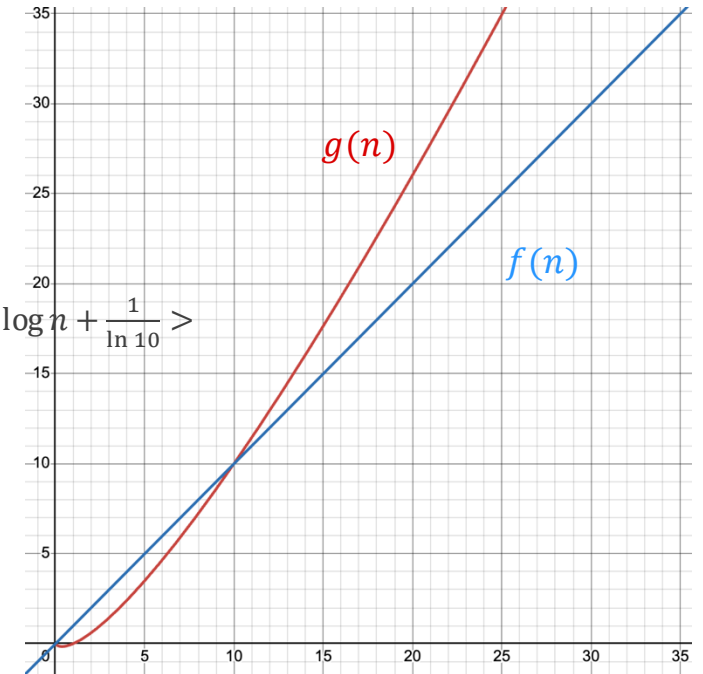
► $\lim_{n \rightarrow \infty} \frac{1}{\log n + \frac{1}{\ln 10}} = 0$. Indeed, the limit exists. By Lemma 4, $f(n) = O(g(n))$

► Definition:

► Consider $c = 1, n_0 = 20$ (many other options)

► Suppose $n \geq 20$ is arbitrary. We will prove that $n \leq n \log n$. It suffices to show that $n \log n - n \geq 0$.

► $n \log n - n = n(\log n - 1)$. Since n is non-negative and $\log n \geq \log 20 \geq 1$ ($\frac{d}{dn}(\log n) = \frac{1}{n \ln 10} > 0 \forall n$, so always increasing), indeed $n \log n - n \geq 0$ because the product of 2 non-negative numbers is always non-negative.



Example of Big-Ω

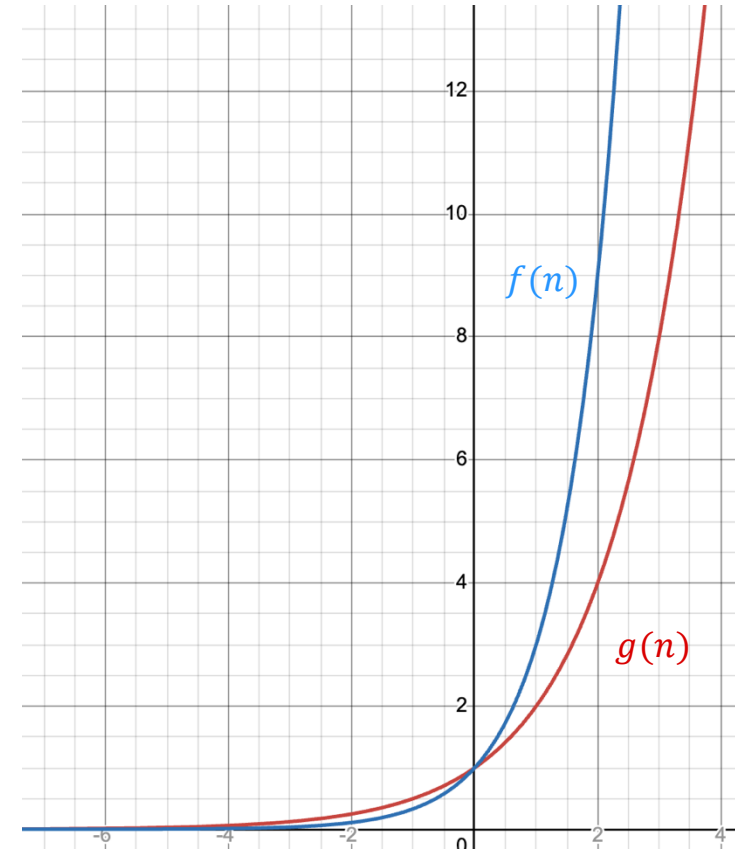
$$f(n) = 3^n, g(n) = 2^n$$

► Limit argument:

► $\lim_{n \rightarrow \infty} \frac{3^n}{2^n} = \lim_{n \rightarrow \infty} \left(\frac{3}{2}\right)^n = \infty$. Indeed, the limit is infinity. By Lemma 5, $f(n) = \Omega(g(n))$

► Definition:

- Consider $c = 1, n_0 = 0$ (many other options)
- Suppose $n \geq 0$ is arbitrary. We will prove that $3^n \geq 2^n$.
- The inequality holds at $n_0 = 0$, since $3^0 = 1$ and $2^0 = 1$.
- Beyond this threshold, 3^n increases more quickly than 2^n . When n increases by 1, 3^n increases by a factor of 3 while 2^n increases by a factor of 2. Thus, the inequality continues to hold for $n \geq n_0$, as needed.



Example of Big- Θ

$$f(n) = n + \sin n^2, g(n) = n$$

► Limit argument:

- $\lim_{n \rightarrow \infty} \frac{n + \sin n^2}{n} = \lim_{n \rightarrow \infty} \left(1 + \frac{\sin n^2}{n}\right) = 1$ (see the Squeeze Theorem). Indeed, the limit exists, is finite, and greater than 0. By Lemmas 4 and 5, $f(n) = \Theta(g(n))$

► Definition:

- Consider $c_1 = \frac{1}{2}, c_2 = 2, n_0 = 5$
- Suppose $n \geq 5$ is arbitrary. We will prove that $\frac{1}{2}n \leq n + \sin n^2 \leq 2n$.
- We will first prove that $n + \sin n^2 \geq \frac{1}{2}n$. By the definition of the $\sin n^2$ function, $n + \sin n^2 \geq n - 1 \geq n - \frac{1}{2}n = \frac{1}{2}n$ given that $1 \leq \frac{1}{2}n \forall n$.
- We will then prove that $n + \sin n^2 \leq 2n$. By the definition of the $\sin n^2$ function, $n + \sin n^2 \leq n + 1 \leq n + n = 2n$ given that $1 \leq n \forall n$.
- By the definition of Big- Θ , $f(n) = \Theta(g(n))$

